

Instituto Tecnológico de Costa Rica
Escuela Ingeniería en Computadores
Curso: CE1101 - Introducción a la
Programación

Documentación técnica

OvercookTEC

Estudiantes:

Alexa Herrera Sánchez

Cristopher Víquez Quirós

Profesor:

Leonardo Araya Martínez

Semestre I 2026

0. Índice

0. Índice.....	2
1. Introducción.....	3
1.1. Propósito del documento.....	3
1.2. Descripción del proyecto.....	3
1.3. Objetivos y Alcance.....	3
2. Descripción General del Sistema.....	4
2.1. Perspectiva del Producto.....	4
2.2. Funcionalidades principales.....	5
2.3. Restricciones del sistema, Supuestos y Dependencias.....	5
3. Requisitos del Sistema.....	6
3.1. Requisitos Funcionales.....	6
3.2. Requisitos No Funcionales.....	7
4. Diseño de la Solución.....	8
4.1. Decisiones de Diseño y Justificación.....	8
4.2. Diagramas.....	8
5. Implementación y Validación de Calidad.....	10
5.1. Estructura del repositorio.....	10
5.2. Estrategias y Casos de Prueba.....	11
5.3. Resultados de pruebas y problemas encontrados.....	11
6. Declaración de uso de Inteligencia Artificial.....	11
6.1. Herramientas de IA utilizadas.....	11
6.2. Herramientas de IA utilizadas.....	12
6.3. Declaración de Responsabilidad y Autoría.....	12
7. Conclusión.....	13
8. Bibliografía.....	14

1. Introducción

Durante esta documentación se describirá la estructuración, diseño e implementación del proyecto OvercookTEC, un videojuego de cocina desarrollado en Python con la biblioteca Pygame. El proyecto está inspirado en la dinámica del juego Overcooked de Nintendo, y fue adaptado como una propuesta educativa dentro del contexto del curso de Introducción a la Programación. El objetivo principal es demostrar la aplicación práctica de los principios de programación orientada a objetos en un entorno interactivo y entretenido.

1.1. Propósito del documento

Este documento tiene como finalidad hacer una entrega formal del Proyecto #2 del curso CE1101, presentando de manera clara y estructurada la arquitectura, diseño e implementación del sistema Crazy Snack Rush. Va dirigido estrictamente al Instituto Tecnológico de Costa Rica, específicamente a la Escuela de Ingeniería en Computadores, para la asignatura de Introducción a la Programación (CE1101).

1.2. Descripción del proyecto

El sistema de código de este juego interactivo se basa en una serie de comandos de todo tipo, funciones, llamadas, y bibliotecas utilizadas por medio de Visual Studio Code, usando el lenguaje Python, y la biblioteca Pygame como recurso indispensable para aspectos cruciales en la creación del proyecto (claro es el ejemplo de la parte funcional y visual del juego). El juego coloca al jugador en el rol de cocinero en un restaurante de snacks, con la misión de preparar y entregar la mayor cantidad de recetas posible dentro del tiempo límite de cada nivel. El programa cuenta con tres niveles de dificultad progresiva, cada uno con su propia cocina, recetas y desafíos. El usuario final previsto es el público joven, aunque la dinámica es accesible para cualquier persona con interés en videojuegos casuales.

1.3. Objetivos y Alcance

El alcance de este proyecto se limita al desarrollo de un videojuego de escritorio con tres escenarios o niveles, dos chefs controlables por teclado, un sistema de recetas con temporizadores y puntuación, y una interfaz gráfica implementada con Pygame. No solo se busca un aprendizaje respecto a las bases de la programación básica de Python, sino también a

las múltiples posibilidades que sus recursos relacionados permiten (tal es el caso de las librerías externas).

Como objetivo principal se busca la solución de problemas y el uso de la creatividad mediante herramientas de programación para desarrollar un videojuego de cocina funcional en Python aplicando correctamente los principios de programación orientada a objetos, que cumpla con todos los requisitos establecidos. Respecto a esto, encontramos objetivos específicos:

- Implementar las clases base del sistema (Cocina, Chef, Estación, Ingrediente, Receta) aplicando encapsulamiento, herencia y polimorfismo.
- Generar conocimiento y experiencia con librerías externas (en este caso Pygame), como herramienta para el desarrollo del juego solicitado.
- Implementar la creatividad y la funcionalidad como recursos mediante el lenguaje de programación solicitado.
- Diseñar y desarrollar tres niveles de juego con dificultad progresiva, incluyendo generación aleatoria de recetas y temporizadores.
- Implementar un sistema de puntuación con penalizaciones por recetas expiradas y una pantalla final de resultados por nivel.

2. Descripción General del Sistema

El código presenta en su mayoría funciones, sin embargo también se importaron algunas bibliotecas y se les utilizó objetivamente, asimismo, se aplicaron comandos simples y algunos más complejos para lograr el resultado final. Entre las restricciones que sigue la asignación encontramos la cantidad de recetas, chefs, el listado de puntajes, y el sistema de la cocina que constituye el juego como tal.

2.1. Perspectiva del Producto

El producto opera mediante un sistema de código de lenguaje Python usando la aplicación de Microsoft Visual Studio Code. Es un sistema autónomo de videojuego de escritorio. El sistema se apoya en la biblioteca Pygame para toda la capa gráfica, manejo de eventos y temporización, y en el módulo estándar de Python para lógica aleatoria (random) y control del sistema (sys). Los recursos externos utilizados son únicamente los sprites y sonidos almacenados localmente dentro de la carpeta del proyecto. Esto permitió la simplificación y optimización del código, así como un mejor funcionamiento y desarrollo de los objetivos del proyecto.

2.2. Funcionalidades principales

- Función 1 - Gestión de escenas: El sistema administra seis escenas distintas (Menú, Nivel 1, Nivel 2, Nivel 3, Pantalla de Resultados y Pantalla Final) con transiciones controladas por el estado del juego.
- Función 2 - Control de chefs por teclado: El jugador puede controlar dos chefs y alternar entre ellos usando teclas definidas, moviéndose libremente por la cuadrícula de la cocina.
- Función 3 - Sistema de estaciones de trabajo: La cocina incluye estaciones de despensa, entrega, tabla de picar, cocina / horno, mezcladora y licuadora. Cada una procesa tipos específicos de ingredientes.
- Función 4 - Generación aleatoria de recetas: Las recetas se generan a intervalos aleatorios definidos en configuracion.py, con hasta 3 órdenes activas simultáneamente.
- Función 5 - Registro de datos: El juego registra los datos de los puntos de los jugadores en un archivo, y se muestra en pantalla el top 10 de los mejores puntajes.
- Función 6 - Sistema de temporizadores: Cada nivel tiene un tiempo límite (200, 230 y 250 segundos respectivamente). Cada receta tiene su propio contador de expiración.
- Función 7 - Sistema de puntuación: Se otorgan puntos por ingrediente al entregar recetas. Si una receta expira, los puntos se reducen a la mitad. Tras 3 expiraciones los puntos se anulan y se descuentan del puntaje total.
- Función 8 - Pantalla de resultados por nivel y pantalla final: Al terminar cada nivel se muestra un resumen del puntaje. Al finalizar el tercer nivel se presenta la pantalla final con los puntajes acumulados de los tres niveles.

2.3. Restricciones del sistema, Supuestos y Dependencias

- Restricción 1 - Compatibilidad: El sistema debe ser compatible con Microsoft, ya que se necesita el uso de Visual Studio Code para ejecutar el programa. Además, el sistema requiere Python 3.x y Pygame 2.x instalados en el equipo. No es compatible con entornos web ni dispositivos móviles.

- Restricción 2 - Resolución de pantalla: El juego funciona a una resolución fija de 1008 × 576 píxeles. No se adapta dinámicamente a otras resoluciones.
- Restricción 3 - Monojugador: El juego está diseñado para un único jugador que controla dos chefs. No existe modo multijugador.
- Restricción 4 - Máximo de órdenes activas: El sistema solo puede manejar 3 recetas activas de forma simultánea.
- Supuesto 1 – Versión del sistema: Se asume que el usuario tiene Python 3.8 o superior y Pygame 2.0 o superior instalados correctamente.
- Supuesto 2 - Servidor útil: Se dispone de un servidor con las especificaciones mínimas indicadas.
- Supuesto 3 – Recursos gráficos Se infiere que los archivos de sprites e íconos están presentes en el directorio del proyecto.
- Dependencia 1 – Visual Studio Code: Se necesita esta aplicación, o una similar que pueda correr el código correctamente.
- Dependencia 2 - Librerías: Se depende de la importación de librerías para el funcionamiento del código: Pygame, Sys, Random.

3. Requisitos del Sistema

Esta sección documenta de forma estructurada todos los requisitos que el sistema OvercookTEC debe cumplir, derivados directamente de la estructura del proyecto y de las funcionalidades implementada.

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	El juego debe mostrar un menú principal con opción de iniciar partida.	Alta	Al abrir el juego aparece la pantalla de menú y al presionar la tecla correspondiente inicia el Nivel 1.
RF-002	El jugador debe poder controlar dos chefs por teclado y alternar entre ellos.	Alta	Los chefs se mueven correctamente con las teclas asignadas y se puede intercambiar el chef activo.

RF-003	La cocina debe generar recetas de manera aleatoria a intervalos definidos durante el nivel.	Alta	Las recetas aparecen aleatoriamente entre 15 y 30 segundos de intervalo, con máximo 3 activas.
RF-004	El chef debe poder interactuar con las estaciones de trabajo para procesar ingredientes.	Alta	Al colocarse frente a una estación y presionar acción, el ingrediente sostenido es procesado correctamente.
RF-005	El sistema debe validar que los ingredientes de una receta estén correctamente preparados antes de aceptar la entrega.	Alta	Solo se acepta la entrega si todos los ingredientes están en el estado correcto.
RF-006	Cada receta debe tener un temporizador propio; al expirar, los puntos se reducen a la mitad.	Alta	Tras cada expiración los puntos de la receta se dividen a la mitad. Tras 3 expiraciones, la receta se elimina y se descuentan puntos
RF-007	El puntaje mínimo posible del jugador debe ser cero.	Media	El puntaje total nunca cae por debajo de cero aunque se descuenten penalizaciones.
RF-008	El juego debe contar con tres niveles de dificultad progresiva.	Alta	El jugador transita por Nivel 1, Nivel 2 y Nivel 3, cada uno con tiempos distintos (200 / 230 / 250 segundos).
RF-009	Mostrar pantalla de resultados entre niveles y pantalla final al terminar.	Alta	Al acabar cada nivel aparece la pantalla de resultados. Al terminar el tercer nivel se muestra la pantalla final con los 3 puntajes.

3.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Usabilidad	Los controles del juego deben ser intuitivos y aprendibles en menos de dos minutos.	Un usuario sin instrucciones previas puede iniciar y jugar una partida completa sin ayuda externa.

RNF-002	Portabilidad	El juego debe ejecutarse en sistemas operativos (ya sea Windows, Linux, macOS, etc) compatibles con Python 3.8+.	El programa corre correctamente instalando únicamente Python y Pygame en cualquiera de los tres SO mencionados
RNF-003	Mantenibilidad	El código debe estar organizado en módulos y seguir los principios de POO para facilitar modificaciones futuras.	Las clases están separadas en archivos propios. Agregar una nueva estación no requiere modificar más de dos archivos.
RNF-004	Confiabilidad	El juego no debe cerrarse inesperadamente durante una sesión normal de juego.	Cero crashes en sesiones de prueba de 30 minutos continuas en hardware objetivo.

4. Diseño de la Solución

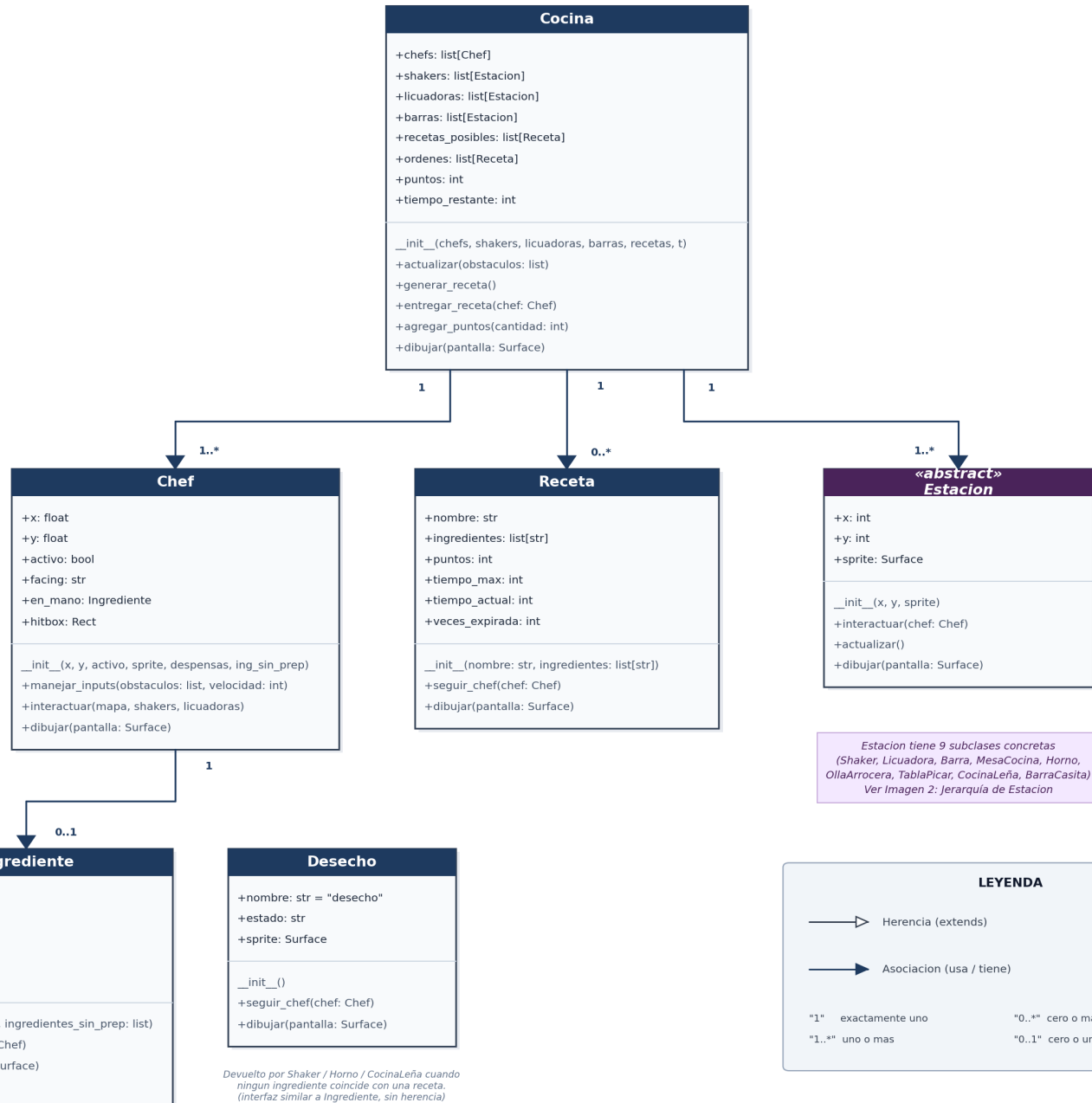
Para el diseño del proyecto se implementaron comandos y funciones específicamente funcionales con librerías elegidas, sin embargo, la simplificación del código fue usada en la medida de lo posible; aun así, se utilizaron componentes con un poco nivel más de complejidad para lograr el resultado deseado. Se mantuvo el orden de la estructuración como herramienta para comprender y ejecutar un mejor trabajo a la hora de realizar el juego.

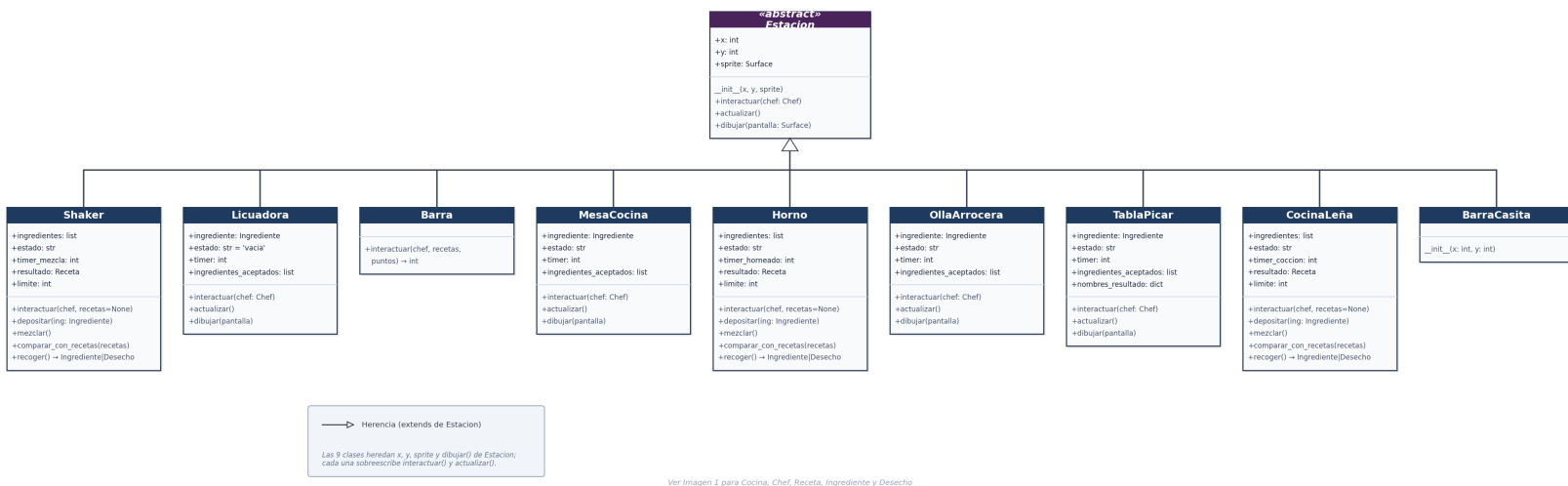
4.1. Decisiones de Diseño y Justificación

En su totalidad, las decisiones de diseño fueron tomadas respecto a los consejos dados por las figuras de enseñanza, y al documento con los requisitos que se nos fueron asignados. Se busco la simplificación mayoritaria como una reducción al margen de error y a una mejor eficiencia en el desarrollo objetivo y funcional del proyecto.

4.2. Digramas UML

(1/2) Núcleo del Dominio





5. Implementación y Validación de Calidad

El proyecto fue sometido a múltiples pruebas para validar su funcionamiento y calidad, no solo en temas de diseño visual, sino también de estructura en cuanto a algoritmos y a su correcto funcionamiento. El repositorio realizado fue de gran utilidad para registrar los errores, cambios, y soluciones que se fueron realizando en el camino, para así poder comprender mejor y facilitar el desarrollo posterior.

5.1. Estructura del repositorio

El repositorio del código se realizó mediante GitHub como fue solicitado en las instrucciones de la asignación, el documento consta de una estructura establecida mediante carpetas:

- / (raíz) → main.py, configuracion.py, README.md.
- /escenas/ → menu.py, nivel1.py, nivel2.py, nivel3.py, resultados.py, pantalla_final.py
- /clases/ → cocina.py, chef.py, estacion.py, ingrediente.py, receta.py (y subclases de Ingrediente).
- /sprites/ → Recursos gráficos en formato PNG (chefs, ingredientes, estaciones, ícono).
- /sonidos/ → Pistas de audio en formato MP3

Los commits del repositorio están organizados en etapas: inicialización del proyecto, implementación de clases base, desarrollo de niveles, integración de interfaz gráfica, y revisión y correcciones finales.

5.2. Estrategias y Casos de Prueba

Durante la realización de este proyecto se utilizaron estrategias fundamentales que facilitaron el trabajo, tales como usar algoritmos sencillos y compatibles con múltiples funcionalidades para seguir una misma “rama” o “tendencia”, lo cual permite un código más conciso y fácil de comprender. Además, se buscaba simplificar y buscar alternativas sencillas en cada caso que fuera posible. A menudo durante la escritura del código se iban realizando pruebas para comprobar que corriera correctamente, así como para ver la diferencia entre algoritmos para elegir cuales eran más convenientes para utilizar.

5.3. Resultados de pruebas y problemas encontrados

Después de múltiples pruebas y comparaciones, se logró llegar a un resultado funcional y bien estructurado, sin embargo en el proceso se dieron problemas encontrados, como es el caso de funciones que no eran “compatibles” con los algoritmos utilizados previamente, o comandos que se estaban usando de manera incorrecta. Al ser nuestra primera vez utilizando Pygame enfrentamos nuevos retos ya que teníamos que entender y aprender su funcionalidad para lograr el producto final deseado, por lo cual el análisis, la investigación, y la ayuda de la inteligencia artificial fue crucial para encontrar estos errores entre tantas líneas de código con funcionamientos distintos, y aún más para otorgarles soluciones optimas y acertadas.

6. Declaración de uso de Inteligencia Artificial y herramientas

Principio de Transparencia Académica

El uso de herramientas de Inteligencia Artificial en proyectos académicos requiere declaración explícita y responsable. Esta sección documenta de forma honesta cómo, cuándo y para qué se utilizaron herramientas de IA en el desarrollo de este proyecto, siguiendo los principios de integridad académica.

6.1. Herramientas de IA utilizadas

Herramienta de IA	Propósito
Copilot	Generación de ideas, redacción de comentarios de código, resolución de dudas técnicas.
Claude	Autocompletado y sugerencias de código durante el desarrollo. Además de revisión de errores y posibles soluciones.

6.2. Descripción del Uso de Inteligencia Artificial

La inteligencia artificial fue utilizada en este proyecto principalmente como una herramienta de revisión de errores y sus posibles soluciones, extendiendo las opciones y la funcionalidad de cada una para una mejor comprensión y un mejor análisis a la hora de elegir que camino tomar durante la resolución de conflictos. Además, fue útil en dudas de diseño visual, ya que se permitió un resultado más limpio y estructurado visualmente.

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Yo, Alexa Herrera Sánchez, quién realice el proyecto "OvercookTEC" declaro que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto es resultado de mi trabajo y esfuerzo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido.
4. Asumo plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

7. Conclusión

El proyecto OvercookTEC cumplió con lo que se buscaba desde un inicio: crear un juego completo y funcional aplicando los conceptos vistos en el curso CE1101. El resultado final es un videojuego que incorpora distintas pantallas, control de chefs por teclado, estaciones de cocina, recetas generadas al azar y un sistema de puntuación con penalizaciones, tal como lo pedían las instrucciones del proyecto.

Uno de los aprendizajes más importantes de este proyecto fue entender cómo organizar el código en clases que se relacionan entre sí de forma lógica, en lugar de escribir todo de manera suelta. Diseñar los distintos tipos de ingredientes como variaciones de una misma idea general, y separar cada pantalla del juego en su propia clase, ayudó muchísimo a mantener el proyecto ordenado y a poder seguir agregando cosas nuevas sin tener que reescribir lo que ya funcionaba. El desarrollo no fue lineal: surgieron varios problemas los cuales tomó tiempo y varias pruebas resolver, pero también ayudó a entender mejor cómo funciona el juego por dentro y a tener más cuidado al planear cada parte antes de programarla.

A futuro, este proyecto se podría seguir mejorando agregando un modo para dos jugadores, más niveles, mejores efectos de sonido o un sistema que guarde los puntajes más altos. Como el código ya está organizado de forma clara, agregar estas ideas no debería significar empezar de cero. En resumen, OvercookTEC fue un proyecto que permitió poner en práctica lo aprendido en clase de una forma entretenida y retadora. Más allá del resultado final, el proceso de construirlo, equivocarse y corregir errores fue lo que realmente ayudó a entender mejor cómo se desarrolla un programa de este tamaño.

8. Bibliografía

- [1] Documentación_Externa.pdf , Sergio Daniel Álvarez Chanto (Tutor)
- [2] Google, Microsoft Copilot (Búsqueda de ideas, sinónimos y palabras)
- [3] Proyecto OvercookTEC, Leonardo Andrés Araya Martínez (Profesor)
- [4] OvercookTec, Repositorio de GitHub, Cristopher Víquez Quirós y Alexa Herrera Sánchez

Link: <https://github.com/crisviquez/OvercookTEC>